
Verbindung von Virtual Shared Spaces über weite Entfernungen Qualitätssicherung



Erstellt von:

Karlsruhe
12.08.2018

Inhaltsverzeichnis

1	Einleitung	3
2	Testsszenarien	4
2.1	Testszenario 1 - Verbindung als Host	4
2.2	Testszenario 2 - Verbindung als Gast	4
2.3	Testszenario 3 - Simulationsmodell auswählen	4
2.4	Testszenario 4 - Mehrere Benutzer	5
2.5	Testszenario 5 - Kollisionserkennung	5
2.6	Testszenario 6 - Overlay	6
3	Fehlerbeschreibung	7
3.1	PTV VISSIM Kommunikation	7
3.2	Animator	8
3.3	Overlay	9
3.4	VR	10
4	Sonstige Änderungen	11
4.1	Graphische Oberfläche	11
4.2	Animator	12
	Glossar	13

1 Einleitung

In diesem Dokument wird die Qualitätssicherung des PSE-Projekts Virtual Shared Spaces über weite Entfernungen beschrieben. Diese Phase beschäftigt sich mit der Korrektheit und der Stabilität des in der Implementierungsphase entstandenen Codes. Dabei werden die Tests und Testszenarien aus den vorhergehenden Phasen abgearbeitet um festzustellen, dass diese korrekt ablaufen. Gegebenenfalls aufgetretene Fehler werden behoben. Dazu kommt eine ausführliche Beschreibung der Fehler und die Vorgehensweise bei der Behebung.

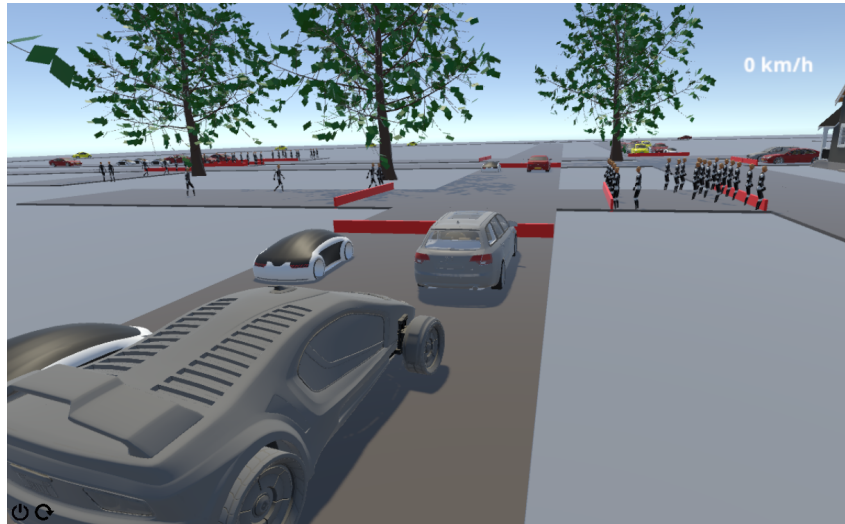


Abbildung 1: Simulationsumgebung

2 Testsszenarien

2.1 Testszenario 1 - Verbindung als Host

2.1.1 Benutzer verbindet sich als Host ohne PTV VISSIM Lizenz

- Start der Anwendung
- Auswahl von Host und Car in der Graphische Benutzeroberfläche
- Das Fahrzeug wird in die Simulation geladen und es laden keine PTV VISSIM Modelle

2.1.2 Benutzer verbindet sich als Host mit PTV VISSIM Lizenz

- Start der Anwendung
- Auswahl von Host und Car in der Graphische Benutzeroberfläche
- Das Fahrzeug wird in die Simulation geladen und es laden PTV VISSIM Modelle

2.2 Testszenario 2 - Verbindung als Gast

2.2.1 Verbindung als Gast mit richtiger IP-Adresse

- Start der Anwendung
- Eingabe einer richtigen IP-Adresse
- Drücken von "Connect"
- Verbindung mit Host wird aufgebaut
- Das Benutzermodell wird in die Simulation des Hosts gesetzt

2.2.2 Verbindung als Gast mit falscher IP-Adresse

- Start der Anwendung
- Eingabe einer falschen IP-Adresse
- Drücken von "Connect"
- Verbindung mit Host schlägt fehl
- Benutzer verbleibt bei der graphische Benutzeroberfläche

2.3 Testszenario 3 - Simulationsmodell auswählen

2.3.1 Benutzer wählt Car als Simulationsmodell

- Start der Anwendung
- Auswahl von Car und Host in der Graphische Benutzeroberfläche
- Der Benutzer wird mit dem Automodell in die Simulation geladen

2.3.2 Benutzer wählt Pedestrian als Simulationsmodell

- Start der Anwendung
- Auswahl von Pedestrian und Host in der Graphische Benutzeroberfläche
- Der Benutzer wird mit dem Fußgängermodell in die Simulation geladen

2.4 Testszenario 4 - Mehrere Benutzer

2.4.1 Mehrere Benutzer Lokal

- Start der Anwendung
- Mindestens 1 Benutzer wählt Host, die anderen Benutzer wählen Gast
- Die Hosts starten ihre Instanz der Simulationsumgebung
- Die Gast-Benutzer geben die IP-Adresse des jeweiligen Hosts ein
- Die Gast-Benutzer wählen "Connect"
- Alle Benutzer befinden sich in den gewählten Instanzen der Simulationsumgebung welche zur angegebenen IP-Adresse gehören

2.4.2 Mehrere Benutzer Internet

- Start der Anwendung
- Der Host drückt "Start Matchmaker"
- Host verändert den "Room Name"
- Host drückt "Create Match"
- Die Gast-Benutzer drücken "Start Matchmaker"
- Gast-Benutzer drücken "List Matches"
- Gast-Benutzer wählen den Raum in "Room to connect"
- Gast-Benutzer drücken "Join Match"
- Alle Benutzer befinden sich in den ausgewählten Simulationsumgebungsinstanzen

2.5 Testszenario 5 - Kollisionserkennung

2.5.1 Kollision von Benutzern und statischen Simulationsobjekten

- Ein Host und ein Gast befinden sich in einer Instanz der Simulation
- Beide versuchen sich durch statische Objekte wie Häuser und Bäume zu bewegen
- Beide können diese Objekte nicht durchdringen

2.6 Testszenario 6 - Overlay

2.6.1 Overlay im Matchmaker Match als Host

- Der Host startet eine Instanz der Simulation über den Matchmaker
- Ihm wird im Overlay der von ihm angegebene Raumname angezeigt
- Außerdem wird im Overlay unter "Clients" eine 1 angezeigt, da nur der Host verbunden ist.

2.6.2 Overlay im Matchmaker Match als Gast

- Der Gast-Benutzer verbindet sich über den Matchmaker zu einer von einem Host erstellten Instanz
- Ihm wird im Overlay der vom Host angegebene Raumname angezeigt
- Außerdem wird im Overlay unter "Client" eine 2 angezeigt, da nur der Gast und der Host verbunden sind

2.6.3 Overlay in lokaler Simulation als Host

- Der Host startet eine lokale Instanz der Simulation
- Ihm wird im Overlay seine aktuelle IP-Adresse angezeigt
- Außerdem wird im Overlay unter "Clients" eine 1 angezeigt, da nur der Host verbunden ist.

2.6.4 Overlay in lokaler Simulation als Gast

- Der Gast-Benutzer verbindet sich über die ihm bekannte IP-Adresse des Hosts mit der Simulation
- Ihm wird im Overlay die IP-Adresse des Hosts angezeigt
- Außerdem wird im Overlay unter "Client" eine 2 angezeigt, da nur der Gast und der Host verbunden sind

3 Fehlerbeschreibung

Dieser Abschnitt befasst sich mit Fehlern, welche durch das Testen aufgefallen sind oder noch seit der Implementierungsphase vorhanden waren. Zu jedem Fehler gibt es eine Beschreibung, sowie eine Angabe wie der Fehler gelöst worden ist.

3.1 PTV VISSIM Kommunikation

3.1.1 Übergabe von Autofahrern und Fußgängern an PTV VISSIM

- **Fehler:** Der Versuch alle vorhandenen Simulationsteilnehmer sowohl als Fußgänger, als auch als Autofahrer an PTV VISSIM zu übergeben schlug fehl. Sobald PTV VISSIM startet kam es zum Absturz
- **Lösung:** Das DrivingSimulatorInterface Objekt, das für die Kommunikation mit PTV VISSIM zuständig ist, besitzt die Parameter MaxNumSimulatorVehicles und MaxNumSimulatorPedestrians. Diese hatten den Wert 1 und verhinderten somit die Übergabe von mehr als einem Autofahrer und Fußgänger. Diese Parameter wurden auf den Wert 20 gesetzt. Dadurch wurde es möglich mehrere Autofahrer zu übergeben. Bei der Übergabe von mehreren Fußgängern, kam es weiterhin zum Absturz.

3.1.2 Übergabe von Fußgängern

- **Fehler:** Obwohl der Parameter MaxSimulatorPedestrians des DrivingSimulator Objektes auf 20 stand, war es nicht möglich mehrere Fußgänger an PTV VISSIM zu übergeben. Beim Start von PTV VISSIM kam eine Fehlermeldung und es stürzte ab.
- **Lösung:** Wir konnten nicht ausfindig machen, wo das Problem in unserem Code liegt und vermuten deshalb einen Fehler in der Schnittstelle zu PTV VISSIM . Damit die, von PTV VISSIM, bereitgestellten Algorithmen dennoch auf die Fußgänger reagieren können, werden diese als Autos übergeben.

3.2 Animator

3.2.1 Synchronisierung der Animation zwischen Clients

- **Fehler:** Die Animationen wurden zuerst nur auf der lokalen Seite angezeigt. Sie wurden allerdings nicht mit den anderen Simulationsteilnehmern synchronisiert.
- **Lösung:** Durch hinzufügen einer Netzwerkschnittstelle an den Animator, wurde dieser netzwerkfähig. Dadurch werden die Animationen durch die Unity-Engine korrekt an die Clients übertragen.

3.2.2 Korrekte Darstellung der verschiedenen Animationszustände

- **Fehler:** Die Animationen sollten durch einen Zustandsautomaten modelliert werden. Allerdings ergab sich bei uns das Problem, dass dieser Zustandsautomat nicht mit unseren Eingaben übereinstimmte. Dies bedeutete, dass unsere Zustandswechsel zwar korrekt geloggt wurden, aber der Zustandsautomat nicht auf diese Eingaben reagierte.
- **Lösung:** Gelöst wurde das Problem, durch die Nutzung einer speziell für den Zustandsgraphen vorgesehen Variablen. Diese wurde von dem Zustandsgraphen erkannt, wodurch dieser unsere Animationen fehlerfrei ausführen konnte. Allerdings war dies auch mit mehr Aufwand als ursprünglich geplant verbunden, da wir nun jeden Übergang von Hand einfügen mussten und konfigrierende Variablenbelegungen mit einbeziehen mussten.

3.2.3 Realistische Geschwindigkeiten für die Animationen

- **Fehler:** Zu Anfang waren unsere Animation leider so gewählt, dass einige von ihnen beim Ausführen zu langsam bzw. zu schnell abgespielt wurden. Dadurch wurde der Realismus der Simulation eingeschränkt, da nun ein Unterschied zwischen einem realen Simulationsteilnehmer und einem vom Computer gesteuerten Teilnehmer erkenntlich wurde. Außerdem beeinflusste die Animationsgeschwindigkeit auch noch die Bewegungsgeschwindigkeit des Nutzers. Dies führte dazu, dass sich bei hoher Animationsgeschwindigkeit, auch der Nutzer proportional schneller bewegte.
- **Lösung:** Die Animationsgeschwindigkeit wurde, im Gegensatz zu unseren Erwartungen, auch schon von der Geschwindigkeitsvariable im Zustandsautomaten beeinflusst. Dies bedeutete, dass wir den Zustandsautomaten so anpassen mussten, dass dieser auch die korrekten Geschwindigkeits-Multiplizierer enthält. Diesen Geschwindigkeits-Multiplizierer konnten wir allerdings auch darauf beschränken, dass er nur für die Animationen zuständig blieb und die Bewegungsgeschwindigkeiten des Nutzers nicht beeinflusste.

3.3 Overlay

3.3.1 Host-IP-Anzeige bei lokaler Verbindung

- **Fehler:** Die IP-Adresse im Overlay bei einer lokalen Verbindung hat beim Host stetig “localhost” angezeigt.
- **Lösung:** Bisher wurde die IP-Adresse über das “IP-Adressen Input Feld” in der Graphische Benutzeroberfläche abgerufen. Allerdings verbindet sich der Host immer über “localhost”. Die Lösung war es, über die Methode `Network.player.ipAddress`, die IPv4 bzw. Ipv6 Adresse des Hosts abzurufen.

3.3.2 RoomName-Anzeige

- **Fehler:** Haben sich Host und Client über den Matchmaker miteinander verbunden, wurde ihnen im Overlay die IP-Adresse des Hosts angezeigt, obwohl diese bei der Verbindung mit dem Matchmaker nicht relevant ist.
- **Lösung:** Statt der IP-Adresse wird nun der vom Host gewählte Raumname im Overlay für Client und Host angezeigt.

3.3.3 Clients-Anzeige

- **Fehler:** Haben sich Clients lokal oder über den Matchmaker mit einem Host verbunden, wurde nur dem Host die aktuelle Zahl verbundener Clients angezeigt. Bei den verbundenen Clients wurde dieser Wert nicht aktualisiert.
- **Lösung:** Nur dem Host wird über seinen CustomManager die richtige Anzahl verbundener Clients angezeigt. Den Clients zeigt der CustomManager immer “0” an. Daher musste mithilfe von sogenannten Syncvar Variablen der richtige Wert vom Host zum Client übertragen werden.

3.4 VR

3.4.1 Position der VR Kamera

- **Fehler:** Die Position der VR Kamera ließ sich nicht im Kopf des Modells fixieren. Dadurch befand sich die VR Kamera oberhalb des Modells und konnte sich bewegen, ohne eine Bewegung des Modells. Bei Modellbewegung, machte die VR Kamera die gleiche Bewegung von ihrer Position aus.
- **Lösung:** Die Positionsverfolgung des VR Headsets wurde ausgeschaltet, wodurch sich die VR Kamera immer im Kopf des Modells befindet und nur noch die Rotation des VR Headsets verfolgt wird.

3.4.2 Fußgängerdrehung in VR

- **Fehler:** Die Rotation des VR Headsets ließ sich nicht auf die Blickrichtung des Modells übertragen, wodurch sich die Richtung der Vorwärtsbewegung nicht ändern ließ. Oder das Modell drehte sich unendlich um sich selbst, wenn die Rotation des VR Headsets nicht 0 entsprach.
- **Lösung:** Die Rotation des Modells wird über die Mausbewegung gesteuert. Das Umgucken funktioniert weiterhin durch das VR Headset.

3.4.3 Bewegung mit VR Controller

- **Fehler:** Mit angeschlossenem VR Controller war die Bewegung der Modelle mit dem Touchpad möglich. Allerdings wurde durch eine Bewegung am Playseat eine unendliche Bewegung nach hinten mit maximaler Richtungsänderung nach links ausgelöst.
- **Lösung:** Mit der nicht vorhandenen Möglichkeit das Modell mit dem VR Headset zu drehen, verliert auch der VR Controller seinen Sinn. Somit wurde er wieder aus dem InputManager entfernt.

4 Sonstige Änderungen

4.1 Graphische Oberfläche

4.1.1 Aussehen

Die graphische Benutzeroberfläche wurde von uns überholt und vom Aussehen modernisiert. Außerdem ist nun eine weitere Auswahlmöglichkeit hinzugekommen, welche die VR-Funktionalität bei Wunsch ein oder ausschalten lässt.

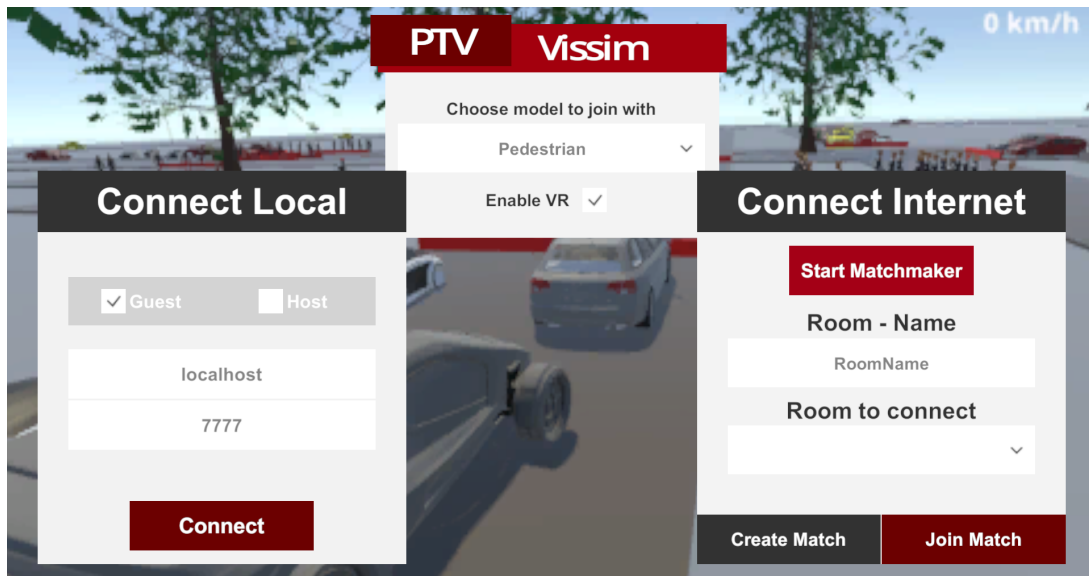


Abbildung 2: neue graphische Benutzeroberfläche

4.1.2 Zurück zur graphischen Oberfläche

Mit der Tasteneingabe “Escape”, hatte sich bisher die Simulation geschlossen. Nun kehrt man zur graphische Benutzeroberfläche zurück und kann sich erneut verbinden oder eine Simulation starten.

4.2 Animator

4.2.1 Gesten

Der Zustandsautomat, welcher für die Animation zuständig ist, erhielt noch weitere Zustände für Gesten. Die zugehörigen Animationen wurden aus dem Asset Store von Unity runtergeladen und auf unsere Modelle angepasst.

4.2.2 Idle Animation

Die Idle-Animation wurde dahingehend überarbeitet, dass diese nun realistischer wirkt.

4.2.3 Laufanimation

Neben der verbesserten Idle-Animation wurde auch eine Renn- und Laufanimation eingefügt welche der Animation der anderen, von VISSIM gesteuerten, Fußgänger entspricht.

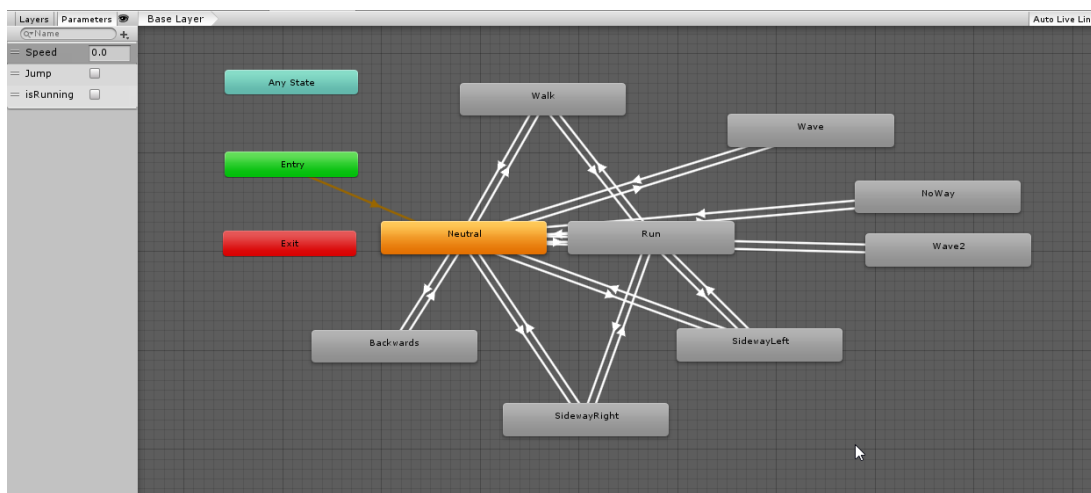


Abbildung 3: Die Motion State Machine des Animators

Glossar

Client Benutzer, der sich auf einem Server befindet.

Gast Der Benutzer, der dem Server eines anderen Benutzers beitrifft.

graphische Benutzeroberfläche Eine graphische Benutzeroberfläche erleichtert dem Benutzer das Nutzen von Funktionen des Anwendungsdiagramms.

Host Der Benutzer, der einen Server mit gültiger PTV VISSIM Lizenz bereitstellt.

IP-Adresse Eine IP-Adresse ist eine Adresse in Computernetzen, die auf dem Internet Protocol (IP) basiert.

Matchmaker Verbindungsoption des Network Managers, in der der Host einen Raum erstellt, dem andere Benutzer beitreten können.

PTV VISSIM Eine Verkehrsfluss-Simulationssoftware der Firma PTV.

Unity Unity ist eine Laufzeit- und Entwicklungsumgebung für Spiele (Spiel-Engine) des Unternehmens Unity Technologies.

Virtual Shared Spaces Virtuelle Räume, in denen mehrere Benutzer gleichzeitig miteinander interagieren können.